

Hiver SDE Intern Assignment: AI Support Agent Report

1. Problem Framing

For AppleSupport, "good" means an agent that can reliably parse unstructured, often frustrated tweets and decide whether an automated reply with a support article is sufficient, or if a human needs to intervene. The dataset is noisy (slang, typos, multi-part tweets).

What I chose not to build:

1. **Multi-turn Context Memory:** The pipeline evaluates only the first customer message to keep the classification latency low and focus on initial triage accuracy.
2. **Fine-tuned LLM:** Instead of fine-tuning a BERT or LLaMA model, I used a zero-shot Prompted LLM (Google Gemini). This allowed for rapid iteration and transparent reasoning outputs, although it sacrifices some throughput speed.

2. Results vs. Baselines

The evaluation was run on a golden set of 150 stratified conversations.

2.1 Classification Metrics

Model	Accuracy	Macro F1
Trivial Baseline (always predicts majority class <code>device_issue</code>)	17.3%	0.037
Keyword Baseline (regex-based heuristics)	59.3%	0.332
AI System (Gemini LLM)	40.7%	0.089

Note: The AI system underperformed the keyword baseline on exact categorical matches because it heavily overpredicted the `other` and `update_software` categories for ambiguous tweets, whereas keywords were strictly tuned to the golden set.

2.2 Escalation Metrics (Should Escalate?)

Model	Precision	Recall	F1 Score
Trivial (never escalate)	0.000	0.000	0.000
Simple Rule (sensitive intent OR low conf)	0.031	0.500	0.058
AI System (6-signal weighted score)	0.029	1.000	0.056

Note: The AI system has a 100% recall (it catches all necessary escalations) but suffers from high false positives (precision 2.9%), meaning it is overly cautious.

2.3 Generation Quality (LLM-as-a-Judge)

An independent LLM judge evaluated a sample of 50 generated replies on a 1-5 scale:

- **Overall Quality:** 3.00 / 5.0

- **Groundedness:** 3.00 / 5.0
- **Correctness:** 3.00 / 5.0
- **Judge-Human Agreement (Cohen's Kappa):** 1.000 (Tested against synthetic human labels).

3. Failure Analysis

Top failure modes identified during evaluation:

1. **Over-cautious Escalation:** The model hallucinates high urgency for standard technical issues (e.g., "my phone is broken" -> Escalate). *Hypothesis: The weights for negative sentiment are set too aggressively.*
2. **"Other" Category Bias:** The classifier defaulted to `other` 34 times when the true intent was `update_software`. *Hypothesis: The prompt definition for software updates was too narrow.*
3. **Missing Context:** Replies for "yes I tried that" are poor. *Hypothesis: Because the system only looks at the first message, it lacks the context of what was previously tried.*
4. **Retrieval Misses on Typos:** BM25 handles typos poorly, and the embedding model sometimes grouped tweets by sentiment rather than technical issue.
5. **Formatting Hallucinations:** The generated replies sometimes included placeholders like `[Link]` instead of the actual retrieved URL.

4. What is misleading about my headline number?

The **100% Recall** for escalation is highly misleading. It sounds perfect, but it is achieved by escalating almost everything (93.3% predicted escalation rate). A system that escalates 93% of tickets provides zero triage value. Additionally, the **40.7% Classification Accuracy** is misleadingly low because the `other` category serves as a dumping ground for complex queries. Many classifications labeled as "incorrect" were semantically valid (e.g., classifying a frozen screen as `device_issue` instead of `update_software`).

5. What you'd do next with one more week

1. **Calibrate Escalation Thresholds:** I would tune the escalation thresholds using a precision-recall curve on a validation set to dramatically reduce the false positive rate.
2. **Few-Shot Prompting:** I would provide the LLM classifier with 3-5 hard negative examples in the prompt to prevent it from defaulting to `other`.
3. **Include Conversational History:** Implement a sliding window of the last 3 messages to resolve context issues.
4. **Deploy as an API:** Wrap the pipeline in FastAPI and containerize it with Docker for a production-ready mock deployment.